22AIE205-Introduction to Python

Lab Exercise 4

Name: AKEPATI SATHISH REDDY

Reg.no: CH.SC.U4AIE25001

Department: AIE(25-29)

Problem Statement 1:

A supermarket records purchases made by multiple customers. For each customer, collect:

- **Customer ID**
- **Customer Name**
- **PAN Number**
- **Purchased Items (multiple items)**
- **Quantity of each item**
- **Unit Price of each item**

Requirements

1. **Calculate the total bill for each customer.**
2. **Apply the following discounts:**
    - **5% if the bill exceeds ₹5,000.**
    - **Additional 3% if the customer purchases more than 10 items.**
    - **If the PAN number starts with A, provide a cashback of ₹8 for every item purchased.**
3. **Store customer information using appropriate Python data structures.**
4. **Identify:**
    - **Customer with the highest bill.**
    - **Most frequently purchased item.**
    - **Item purchased by the maximum number of unique customers.**
    - **Customers who purchased exactly the same set of items.**
5. **Generate a final report showing:**
    - **Total sales**
    - **Average customer bill**
    - **Total discount provided**
    - **Top 3 customers based on bill amount**

Algorithm:

1. Start the program.
2. Read the number of customers.
3. Read the Customer ID, Customer Name, and PAN Number for each customer.
4. Read the number of purchased items.
5. Read the item name, quantity, and unit price for each item.
6. Calculate the total bill by multiplying quantity with unit price.
7. Apply a 5% discount if the bill exceeds ₹5000.
8. Apply an additional 3% discount if the customer purchases more than 10 items.
9. If the PAN number starts with 'A', provide a cashback of ₹8 for every item purchased.
10. Calculate the final bill after deducting the discount and cashback.
11. Store all customer details and billing information.
12. Find the customer with the highest bill.
13. Find the most frequently purchased item.
14. Find the item purchased by the maximum number of unique customers.
15. Identify customers who purchased exactly the same set of items.
16. Calculate the total sales of the supermarket.
17. Calculate the average customer bill.
18. Calculate the total discount provided.
19. Identify the top three customers based on the final bill amount.
20. Display the final report.
21. Stop.

Source Code:

problem1.py > ...

```python

customer_data = []
item_frequency = {}
unique_customers = {}

Ncustomers = int(input("Enter number of customers: "))

for i in range(Ncustomers):

    print("\nCustomer", i + 1)

    customer_id = input("Enter Customer ID: ")
    customer_name = input("Enter Customer Name: ")
    pan = input("Enter PAN Number: ")

    Nitems = int(input("Enter number of items: "))

    items = []
    quantities = []
    prices = []

    total_bill = 0
    total_items = 0

    for j in range(Nitems):

        print("\nItem", j + 1)

        item = input("Enter Item Name: ")
        quantity = int(input("Enter Quantity: "))
        price = float(input("Enter Unit Price: "))

        items.append(item)
        quantities.append(quantity)
        prices.append(price)

        total_bill += quantity * price
```

```
        total_items += quantity

        if item in item_frequency:
            item_frequency[item] += quantity
        else:
            item_frequency[item] = quantity

        if item not in unique_customers:
            unique_customers[item] = set()

        unique_customers[item].add(customer_name)

    customer = {
        "ID": customer_id,
        "Name": customer_name,
        "PAN": pan,
        "Items": items,
        "Quantity": quantities,
        "Price": prices,
        "Bill": total_bill,
        "TotalItems": total_items
    }

    discount = 0

    if total_bill > 5000:
        discount += total_bill * 0.05

    if total_items > 10:
        discount += total_bill * 0.03

    cashback = 0

    if pan[0] == 'A':
        cashback = total_items * 8

    final_bill = total_bill - discount - cashback

    customer["Discount"] = discount
    customer["Cashback"] = cashback
    customer["FinalBill"] = final_bill

    customer_data.append(customer)

highest_customer = customer_data[0]

for customer in customer_data:
    if customer["FinalBill"] > highest_customer["FinalBill"]:
        highest_customer = customer

most_item = ""
maximum = 0

for item in item_frequency:
    if item_frequency[item] > maximum:
        maximum = item_frequency[item]
        most_item = item

unique_item = ""
maximum_customer = 0

for item in unique_customers:
    if len(unique_customers[item]) > maximum_customer:
        maximum_customer = len(unique_customers[item])
        unique_item = item

same_items = []

for i in range(len(customer_data)):
    for j in range(i + 1, len(customer_data)):
        if set(customer_data[i]["Items"]) == set(customer_data[j]["Items"]):
            same_items.append((customer_data[i]["Name"], customer_data[j]["Name"]))
```

```
total_sales = 0
total_discount = 0
for customer in customer_data:
    total_sales += customer["FinalBill"]
    total_discount += customer["Discount"]

average_bill = total_sales / len(customer_data)

top3 = sorted(customer_data,
              key=lambda x: x["FinalBill"],
              reverse=True)

print("\n========== FINAL REPORT ==========\n")

print("Customer with Highest Bill :", highest_customer["Name"])
print("Most Frequently Purchased Item :", most_item)
print("Item Purchased by Maximum Unique Customers :", unique_item)

print("\nCustomers with Same Set of Items")

if len(same_items) == 0:
    print("None")
else:
    for pair in same_items:
        print(pair[0], "and", pair[1])

print("\nTotal Sales :", total_sales)
print("Average Customer Bill :", average_bill)
print("Total Discount Provided :", total_discount)

print("\nTop 3 Customers")

limit = min(3, len(top3))

for i in range(limit):
    print(i + 1, ".", top3[i]["Name"],
          "-", top3[i]["FinalBill"])
```

Sample Inputs and Outputs:

```
PS D:\SEMESTERs\SEMESTER3\22AIE205-Python\160726Lab> python -u

Enter number of customers: 3

Customer 1
Enter Customer ID: C122
Enter Customer Name: Ramesh
Enter PAN Number: ABCDF4567U
Enter number of items: 2

Item 1
Enter Item Name: Mobile
Enter Quantity: 2
Enter Unit Price: 4000

Item 2
Enter Item Name: Laptop
Enter Quantity: 1
Enter Unit Price: 56000

Customer 2
Enter Customer ID: C167
Enter Customer Name: Fanju
Enter PAN Number: QWERD3678I
Enter number of items: 2

Item 1
Enter Item Name: Cakes
Enter Quantity: 3
Enter Unit Price: 50

Item 2
Enter Item Name: Milk
Enter Quantity: 3
Enter Unit Price: 50

Customer 3
Enter Customer ID: C512
Enter Customer Name: Wenu
Enter PAN Number: ASDFG5215Y
Enter number of items: 2

Item 1
Enter Item Name: Wheat
Enter Quantity: 4
Enter Unit Price: 200

Item 2
Enter Item Name: Laddu
Enter Quantity: 100
Enter Unit Price: 10

========== FINAL REPORT ==========

Customer with Highest Bill : Ramesh
Most Frequently Purchased Item : Laddu
Item Purchased by Maximum Unique Customers : Mobile

Customers with Same Set of Items
None

Total Sales : 61990.0
Average Customer Bill : 20663.333333333332
Total Discount Provided : 3254.0

Top 3 Customers
1 . Ramesh  - 60776.0
2 . Wenu - 914.0
3 . Fanju - 300.0
```

Explanation of the chosen data structures and their justification:

1) **List : Used to store customer details, item names, quantities, and prices because lists can store multiple ordered values and allow modifications.**
2) **Dictionary : Used to store complete customer information and item frequencies because dictionaries provide fast access through key-value pairs.**
3) **Set :Used to store unique customers for each item and compare purchased item sets because sets automatically remove duplicates and support efficient comparison.**
4) **String :Used to store Customer ID, Customer Name, PAN Number, and Item Name because these are text data.**
5) **Integer : Used to store quantities, number of customers, and number of items because these values are numbers.**
6) **Float : Used to store unit price, bill amount, discount, cashback, and final bill because monetary values may contain decimal values.**

Problem Statement 2:

A university wants to analyze examination results of students from different departments.

For each student, collect:

- **Register Number**
- **Name**
- **Department**
- **Marks in five subjects**

Requirements

1. **Compute total, average, percentage, grade, and pass/fail status.**
2. **Store:**
    - **Student identity as a tuple.**
    - **Student records in a dictionary.**
    - **Department-wise students using sets.**
3. **Determine:**
    - **Overall topper.**
    - **Department-wise toppers.**
    - **Students scoring above 90 in at least three subjects.**
    - **Students failing in exactly one subject.**
    - **Subject with the highest class average.**
4. **Detect duplicate student names across departments.**

5. **Generate a department-wise report containing:**
    - **Number of students**
    - **Pass percentage**
    - **Average marks**
    - **Highest and lowest scorer**

Algorithm:

1. **Start the program.**
2. **Read the number of students.**
3. **Read the Register Number, Name, Department, and marks in five subjects for each student.**
4. **Calculate the total marks, average marks, and percentage.**
5. **Assign the grade based on the percentage.**
6. **Determine the pass or fail status based on the marks obtained in all subjects.**
7. **Store the student identity as a tuple.**
8. **Store the complete student record in a dictionary.**
9. **Store department-wise student names using sets.**
10. **Identify the overall topper.**
11. **Identify the topper of each department.**
12. **Find students scoring above 90 in at least three subjects.**
13. **Find students failing in exactly one subject.**
14. **Calculate the average marks of each subject and identify the subject with the highest class average.**
15. **Detect duplicate student names across different departments.**
16. **Generate a department-wise report showing the number of students, pass percentage, average marks, highest scorer, and lowest scorer.**
17. **Display all results.**
18. **Stop the program.**

Source Code:

problem2.py > ...

```python
student_records = {}
department_students = {}
subject_totals = [0, 0, 0, 0, 0]

n = int(input("Enter number of students: "))

for i in range(n):

    print("\nStudent", i + 1)

    reg_no = input("Enter Register Number: ")
    name = input("Enter Name: ")
    department = input("Enter Department: ")

    identity = (reg_no, name, department)

    marks = []
    total = 0
    pass_count = 0
    above90 = 0

    print("Enter marks of 5 subjects")

    for j in range(5):

        mark = int(input("Subject " + str(j + 1) + ": "))

        marks.append(mark)

        total += mark

        subject_totals[j] += mark

        if mark >= 50:
            pass_count += 1

        if mark > 90:
            above90 += 1

    average = total / 5

    percentage = average

    if percentage >= 90:
        grade = "A"
    elif percentage >= 80:
        grade = "B"
    elif percentage >= 70:
        grade = "C"
    elif percentage >= 60:
        grade = "D"
    elif percentage >= 50:
        grade = "E"
    else:
        grade = "F"

    if pass_count == 5:
        status = "Pass"
    else:
        status = "Fail"

    student_records[reg_no] = {
        "Identity": identity,
        "Marks": marks,
        "Total": total,
        "Average": average,
        "Percentage": percentage,
        "Grade": grade,
        "Status": status,
        "Above90": above90,
        "FailSubjects": 5 - pass_count
    }

    if department not in department_students:
```

```
        department_students[department] = set()

    department_students[department].add(reg_no)

overall_topper = ""
highest_total = -1

department_toppers = {}

for reg_no in student_records:

    student = student_records[reg_no]

    if student["Total"] > highest_total:
        highest_total = student["Total"]
        overall_topper = student

    dept = student["Identity"][2]

    if dept not in department_toppers:
        department_toppers[dept] = student

    elif student["Total"] > department_toppers[dept]["Total"]:
        department_toppers[dept] = student

print("\nOverall Topper :", overall_topper["Identity"][1])
print("\nDepartment-wise Toppers")

for dept in department_toppers:
    print(dept, ":", department_toppers[dept]["Identity"][1])

print("\nStudents scoring above 90 in at least three subjects")

for reg_no in student_records:
    if student_records[reg_no]["Above90"] >= 3:
        print(student_records[reg_no]["Identity"][1])
```

```
print("\nStudents failing in exactly one subject")

for reg_no in student_records:
    if student_records[reg_no]["FailSubjects"] == 1:
        print(student_records[reg_no]["Identity"][1])

highest_average = subject_totals[0] / n
subject = 1

for i in range(1, 5):

    average = subject_totals[i] / n

    if average > highest_average:
        highest_average = average
        subject = i + 1

print("\nSubject with Highest Class Average : Subject", subject)

department_report = {}

for dept in department_students:

    department_report[dept] = {
        "Students": 0,
        "Pass": 0,
        "TotalMarks": 0,
        "Highest": -1,
        "Lowest": 501
    }
    names = {}

for reg_no in student_records:

    student = student_records[reg_no]

```

```
    dept = student["Identity"][2]

    department_report[dept]["Students"] += 1
    department_report[dept]["TotalMarks"] += student["Total"]

    if student["Status"] == "Pass":
        department_report[dept]["Pass"] += 1
    if student["Total"] > department_report[dept]["Highest"]:
        department_report[dept]["Highest"] = student["Total"]
    if student["Total"] < department_report[dept]["Lowest"]:
        department_report[dept]["Lowest"] = student["Total"]
    name = student["Identity"][1]
    if name not in names:
        names[name] = set()
    names[name].add(dept)

print("\nDuplicate Student Names Across Departments")

for name in names:
    if len(names[name]) > 1:
        print(name)

print("\nDepartment-wise Report")

for dept in department_report:

    students = department_report[dept]["Students"]

    pass_percentage = (department_report[dept]["Pass"] / students) * 100

    average_marks = department_report[dept]["TotalMarks"] / students

    print("\nDepartment :", dept)
    print("Number of Students :", students)
    print("Pass Percentage :", pass_percentage)
    print("Average Marks :", average_marks)
    print("Highest Scorer :", department_report[dept]["Highest"])
```

Sample Input(s) and Output(s):

```
PS D:\SEMESTERs\SEMESTER3\22AIE205-Python\160726Lab> python -u "d
-Python\160726Lab\problem2.py"
Enter number of students: 3

Student 1
Enter Register Number: 101
Enter Name: Rahul
Enter Department: AIE
Enter marks of 5 subjects
Subject 1: 40
Subject 2: 60
Subject 3: 78
Subject 4: 66
Subject 5: 77

Student 2
Enter Register Number: 102
Enter Name: Sahul Raj
Enter Department: CSE
Enter marks of 5 subjects
Subject 1: 68
Subject 2: 55
Subject 3: 78
Subject 4: 89
Subject 5: 88

Student 3
Enter Register Number: 103
Enter Name: Rashmika
Enter Department: CYS
Enter marks of 5 subjects
Subject 1: 66
Subject 2: 78
Subject 3: 87
Subject 4: 59
Subject 5: 80

Overall Topper : Sahul Raj

Department-wise Toppers
AIE : Rahul
```

```
CSE : Sahul Raj
CYS : Rashmika

Students scoring above 90 in at least three subjects

Students failing in exactly one subject
Rahul

Subject with Highest Class Average : Subject 5

Duplicate Student Names Across Departments

Department-wise Report

Department : AIE
Number of Students : 1
Pass Percentage : 0.0
Average Marks : 321.0
Highest Scorer : 321
Lowest Scorer : 321

Department : CSE
Number of Students : 1
Pass Percentage : 100.0
Average Marks : 378.0
Highest Scorer : 378
Lowest Scorer : 378

Department : CYS
Number of Students : 1
Pass Percentage : 100.0
Average Marks : 370.0
Highest Scorer : 370
Lowest Scorer : 370
```

Explanation of the chosen data structures and their justification:

1. **Tuple :Used to store the student identity (Register Number, Name, Department) because tuples are immutable and suitable for fixed data.**
2. **Dictionary :Used to store complete student records because dictionaries organize data using key-value pairs for easy retrieval.**
3. **Set : Used to store department-wise student names because sets maintain unique values and help detect duplicate names across departments.**
4. **List :Used to store marks of five subjects because lists allow ordered storage and easy access to subject marks.**
5. **String : Used to store Register Number, Name, Department, Grade, and Pass of fail Status because they are text data.**
6. **Integer :Used to store marks and total marks because they are numberd values.**
7. **Float :Used to store average marks and percentage because these values may contain decimal numbers.**

Problem Statement 3:

Students register for elective courses.

For each student collect:

- **Register Number**
- **Name**
- **Department**
- **List of electives selected**

Requirements

1. **A student cannot register for the same course twice.**
2. **Find:**
    - **Most popular elective.**
    - **Least popular elective.**
    - **Students choosing more than three electives.**
    - **Students with identical elective combinations.**
    - **Electives selected by only one student.**
3. **Display department-wise enrollment statistics.**
4. **Recommend one additional elective for every student based on the electives chosen by other students in the same department.**

Display all reports in sorted order.

Algorithm:

1. **Start the program.**
2. **Read the number of students.**
3. **Read the Register Number, Name, Department, and number of electives for each student.**
4. **Read the elective names and store them in a set to avoid duplicate registrations.**
5. **Store the student identity as a tuple and the complete student record in a dictionary.**
6. **Store department-wise student details and count the enrollment of each elective.**
7. **Find the most popular elective based on the highest enrollment count.**
8. **Find the least popular elective based on the lowest enrollment count.**
9. **Identify students who have selected more than three electives.**
10. **Find students who have selected exactly the same combination of electives.**
11. **Identify electives selected by only one student.**
12. **Generate department wise enrollment statistics.**
13. **Recommend one additional elective to each student based on electives chosen by other students in the same department.**
14. **Display all reports in sorted order.**

Source Code:

problem3.py > ...

```python
register_numbers = []
student_names = []
departments = []

student_records = {}
department_students = {}
elective_count = {}

Nstudents = int(input("Enter no.of students: "))

i = 1

while Nstudents >= 1:

    print("Student", i)

    reg = input("Enter Register Number: ")
    name = input("Enter Name: ")
    dept = input("Enter Department: ")

    register_numbers.append(reg)
    student_names.append(name)
    departments.append(dept)

    identity = (reg, name, dept)

    n = int(input("Enter no.of electives: "))

    electives = set()

    item = 1

    while n >= 1:

        course = input("Enter Elective " + str(item) + ": ")

        electives.add(course)

        if course in elective_count:
            elective_count[course] += 1
        else:
            elective_count[course] = 1

        item += 1
        n -= 1

    student_records[reg] = {
        "Identity": identity,
        "Electives": electives
    }

    if dept not in department_students:
        department_students[dept] = []
        department_students[dept] = []

    department_students[dept].append(reg)

    i += 1
    Nstudents -= 1

most_popular = ""
least_popular = ""

maximum = -1
minimum = 1000

for course in elective_count:

    if elective_count[course] > maximum:
        maximum = elective_count[course]
        most_popular = course

    if elective_count[course] < minimum:
        minimum = elective_count[course]
```

```
        least_popular = course

print("\nMost Popular Elective :", most_popular)
print("Least Popular Elective :", least_popular)

print("\nStudents choosing more than three electives")

for reg in student_records:

    if len(student_records[reg]["Electives"]) > 3:
        print(student_records[reg]["Identity"][1])

same_courses = []

regs = list(student_records.keys())

i = 0

while i < len(regs):
    j = i + 1
    while j < len(regs):
        if student_records[regs[i]]["Electives"] == student_records[regs[j]]["Electives"]:
            same_courses.append((regs[i], regs[j]))
        j += 1
    i += 1
    print("\nStudents with Identical Elective Combinations")

if len(same_courses) == 0:
    print("None")
else:
    i = 0
    while i < len(same_courses):
        print(student_records[same_courses[i][0]]["Identity"][1],
              "and",
              student_records[same_courses[i][1]]["Identity"][1])
        i += 1
```

```
print("\nElectives Selected by Only One Student")

for course in elective_count:

    if elective_count[course] == 1:
        print(course)

print("\nDepartment-wise Enrollment Statistics")

for dept in sorted(department_students):

    print(dept, ":", len(department_students[dept]), "Students")

print("\nRecommended Electives")

for reg in student_records:

    dept = student_records[reg]["Identity"][2]

    recommend = ""

    for student in department_students[dept]:

        if student != reg:

            for course in student_records[student]["Electives"]:

                if course not in student_records[reg]["Electives"]:

                    recommend = course
                    break

        if recommend != "":
            break

    if recommend == "":
```

```
            print(student_records[reg]["Identity"][1], "-> No Recommendation")
        else:
            print(student_records[reg]["Identity"][1], "->", recommend)

    print("\nAll Reports in Sorted Order")

    print("\nDepartments")

    for dept in sorted(department_students):
        print(dept)

    print("\nElectives")

    for course in sorted(elective_count):
        print(course)

    print("\nStudents")

    for reg in sorted(student_records):

        print(student_records[reg]["Identity"][1])

    print("\nEnd of Report")
```

Sample Input(s) and Output(s):

```
PS D:\SEMESTERs\SEMESTER3\22AIE205-Python\160726Lab> py
-Python\160726Lab\problem3.py"
Enter no.of students: 3
Student 1
Enter Register Number: 119
Enter Name: Renukesh
Enter Department: AIE
Enter no.of electives: 3
Enter Elective 1: OS
Enter Elective 2: CN
Enter Elective 3: Cloud
Student 2
Enter Register Number: 102
Enter Name: Rama Raju
Enter Department: CSE
Enter no.of electives: 2
Enter Elective 1: Iot
Enter Elective 2: CAyber Security
Student 3
Enter Register Number: 110
Enter Name: Samantha K
Enter Department: AI DS
Enter no.of electives: 4
Enter Elective 1: DBMS
Enter Elective 2: Networking
Enter Elective 3: Data Mining
Enter Elective 4: Quantam AI

Most Popular Elective : OS
Least Popular Elective : OS

Students choosing more than three electives
Samantha K

Students with Identical Elective Combinations

Students with Identical Elective Combinations

Students with Identical Elective Combinations
None
```

```
Electives Selected by Only One Student
OS
CN
Cloud
Iot
CAyber Security
DBMS
Networking
Data Mining
Quantam AI

Department-wise Enrollment Statistics
AI DS : 1 Students
AIE : 1 Students
CSE : 1 Students

Recommended Electives
Renukesh -> No Recommendation
Rama Raju -> No Recommendation
Samantha K -> No Recommendation

All Reports in Sorted Order

Departments
AI DS
AIE
CSE

Electives
CAyber Security
CN
Cloud
DBMS
Data Mining
Iot
Networking
OS
```

```
Quantam AI

Students
Rama Raju
Samantha K
Renukesh

End of Report
```

Explanation of the chosen data structures and their justification:

1. **Tuple :Used to store the student's identity (Register Number, Name, Department) because the data is fixed and should not be modified.**
2. **Dictionary : Used to store student records, elective counts, and department-wise student details because dictionaries provide fast access using key-value pairs.**

3. **Set :used to store electives selected by each student because sets automatically remove duplicate electives and make comparison of elective combinations easier.**
4. **List :used to store register numbers, student names, departments, and department-wise student lists because lists maintain ordered collections of data.**
5. **String :Used to store Register Number, Name, Department, and Elective names because they are textual values.**
6. **Integer :Used to store the number of students, number of electives, and elective enrollment counts because they are whole-number values.**